

Knowledge Representation and Reasoning – Mod. 2

Exercise Sheet 3 & 4: RDF & SPARQL

Gian Carlo Milanese

27 April/4 May 2023

Exercise 1 RDF Triples

Which of the triples from Fig. 1 are valid? If a triple is not valid, explain why. Validity in this exercise means that the RDF terms (IRIs, literals, blank nodes) are in the correct position. It does not necessarily mean that the triples are good examples of how information should be represented.

	Subject	Predicate	Object	Valid?
1	<http://example.org/alice>	rdf:type	foaf:Person	...
2	_:alice	_:Type	_:Person	...
3	ex:bob	rdf:type	foaf:Person	...
4	_:Alice	rdf:type	foaf:Person	...
5	"bob"^^xsd:string	rdf:type	_:Person	...
6	_:alice	rdf:type	"Person"^^xsd:string	...
7	_:bob	rdf:type	_:Person	...
8	_:Bob	"Type"^^xsd:string	foaf:Person	...

Figure 1: Triples for exercise 1

Exercise 2 RDF Graphs

Write down a Turtle document corresponding to the following descriptions, using explicitly typed literals where relevant.

1. Bob is a student. Bob knows Alice. Bob is 22-years old. Bob is from Milan.
2. Alice is a 23-year old student who likes the band Pink Floyd.
3. Nick Mason is a drummer and a member of the band Pink Floyd. His name is “Nicholas” and he was born on 27 January 1944.
4. The band Pink Floyd has a member whose name is “David Gilmour” and who plays the guitar.
5. Bob knows that Alice likes Pink Floyd.
6. Alice is attending a Polyphia concert on 23 May 2023 in Milan.

You can use prefixes, in particular the ones defined below:

```

@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns##> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema##> .
@prefix foaf: <http://xmlns.com/foaf/0.1/> .
@prefix ex: <http://example.org/> .
## continue here

```

Use an online validator such as the following to ensure that your syntax is valid.

- <http://ttl.summerofcode.be/>. Turtle validator.
- <https://www.easyrdf.org/converter>. Here you can convert RDF data between different serializations. An error will be thrown if your syntax is not correct.
- <https://www.w3.org/RDF/Validator/>. Here you can validate RDF data serialized as RDF/XML (you can use the previous converter to convert it to this format). If you select “Triples and Graph” among the Display Result Options you can also see a graph representation of your RDF triples.

Exercise 3 Reification Puzzle

Translate the RDF graph represented in Fig. 2 to natural language.

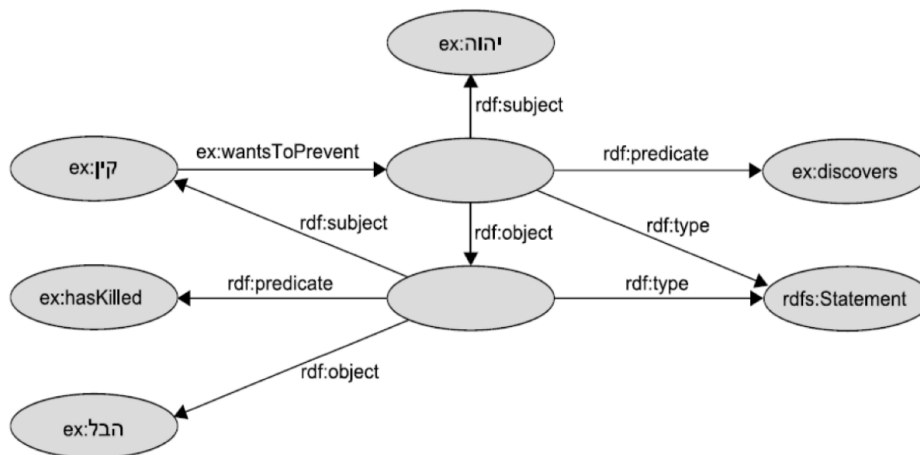


Figure 2: Reification puzzle ([source](#))

Exercise 4 SPARQL Basics

Consider the RDF graph of Fig. 3. Give the results tables for the following queries (prefixes are omitted):

1. `select ?x ?y where { ?x :knows ?y . }`
2. `select ?x where { ?x a :Person . }`
3. `select ?x ?y where { ?x :knows ?y . ?x a :Student . }`
4. `select ?x ?z where { ?x :knows ?y ; a :Student . ?y :name ?z }`
5. `select ?x ?y ?z where { ?x :parentOf ?y . OPTIONAL { ?y :occupation ?z } }`

```

6. select ?x ?y where { ?x rdfs:subClassOf/rdfs:subClassOf ?y }

@prefix : <http://example.org/> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

:bob    :knows :alice .
:carol  :knows :alice .

:alice  :name "Alice" .
:carol  :name "Carol" .

:bob    a :Person .
:carol  a :Student .
:david  a :MathStudent .

:Student rdfs:subClassOf :Person .
:MathStudent rdfs:subClassOf :Student .

:bob    :occupation :Pianist .
:alice  :occupation :Guitarist .

:alice  :parentOf :bob .
:bob    :parentOf :carol .
:carol  :parentOf :david .

:bob :age 49 .
:alice :age 21 .
:carol :age 33 .

```

Figure 3: Sample RDF Graph for queries

Exercise 5 SPARQL Playground

Download [SPARQL Playground](#) and run it (follow the instructions on the page).

Go to “Data”, copy and paste the RDF graph from Fig. 3, click “Reload Turtle Data”, then click on “SPARQL Playground”. You can now query this RDF graph. Try the following queries:

- Retrieve all ?x that know :alice.
- Retrieve all ?x and ?y such that ?x is the parent of ?y
- Retrieve all ?x and ?y such that ?x is the grandparent of ?y
- Retrieve all ?x that know :alice, and their name.
- Retrieve all ?x that know :alice, and their name *if available* (hint: use OPTIONAL).
- Retrieve all ancestors (i.e., parents of parents of parents of parents...) of :david (hint: use property paths).
- Retrieve all ?x of type :Person.
- Retrieve all ?x of type :Person, including subclasses of :Person.
- Retrieve all ?x that are either pianists or guitarists (hint: use UNION).
- Retrieve all ?x that are older than 30 (hint: use FILTER).

Exercise 6 DBpedia: Modifiers

Go to <https://dbpedia.org/sparql>, an online SPARQL endpoint where you can query the DBpedia graph.

A few notes:

- The DBpedia SPARQL endpoint uses some default prefixes for IRIs, so you don't need to specify, e.g., `PREFIX dbo: <http://dbpedia.org/ontology/>` in each query. You can see the default prefixes used by this SPARQL endpoint here: <https://dbpedia.org/sparql/?help=nsdecl>
- If you need to use an IRI and don't know its name, you can try to guess it or Google it (some IRIs will be provided anyway). Note that DBpedia links for resources such as <http://dbpedia.org/resource/Nile> will redirect you to <http://dbpedia.org/page/Nile>. You should use `<http://dbpedia.org/resource/Nile>` (abbreviated `dbr:Nile`) in the queries.

Try the following queries:

1. Retrieve 100 triples
2. Retrieve 100 distinct triples
3. Retrieve 100 distinct predicates.
4. Retrieve 100 people (i.e., resources of type person)
5. Retrieve predicates that can have a resource of type person as subject (some of these predicates are used for later queries)
6. Retrieve the birth names and years of 100 people
7. Retrieve the birth names of 100 people born in 1984, order by name
8. Retrieve the length of the Nile river.
9. Retrieve trumpet players that were also bandleaders.
10. Retrieve movies that were written, directed and produced by the same person

A few useful IRIs:

- | | |
|-------------------------------|-----------------------------|
| • <code>dbo:Person</code> | • <code>dbr:Trumpet</code> |
| • <code>dbo:length</code> | • <code>dbo:Film</code> |
| • <code>dbo:occupation</code> | • <code>dbo:producer</code> |
| • <code>dbr:Bandleader</code> | • <code>dbo:writer</code> |
| • <code>dbo:instrument</code> | |

Exercise 7 DBpedia: Optional Values

Go to <https://dbpedia.org/sparql>. Try the following queries:

1. Retrieve names of writers and, if available, their birth year.
2. The same as the previous query but, if available, retrieve their birth place too.
3. Retrieve movies that were written, directed and produced by the same person. If available, retrieve their box-office gross.

Useful IRIs:

- `dbo:Writer`
- `dbo:birthPlace`
- `rdfs:label` (for names)
- `dbo:Film`

Exercise 8 DBpedia: UNION

Go to <https://dbpedia.org/sparql>. Try the following queries:

1. Retrieve movies that were written, directed or produced by Charlie Kaufman
2. Retrieve people that were born in Milan or in Rome.
3. Same as the previous query, but also retrieve the occupation for people born in Milan.
4. Same as the previous query, but retrieve the occupation for people born in Milan only if available.

Useful IRIs:

- `dbo:Film`
- `dbo:birthPlace`
- `dbr:Milan`

Exercise 9 DBpedia: Property Paths

Go to <https://dbpedia.org/sparql>. Try the following queries:

1. Retrieve movies that were written, directed or produced by Charlie Kaufman
2. Retrieve people that were born or that live in Milan.
3. Retrieve people that have written or directed a movie.
4. Retrieve the names of notable works of Elena Ferrante
5. Retrieve the grandparents of Elizabeth II.
6. Retrieve all ancestors of Elizabeth II.
7. Retrieve the parents, the grandparents, the great-grandparents, and the great-great-grandparents of Elizabeth II.

A few useful IRIs:

- `dbr:Elena_Ferrante`
- `dbo:birthPlace`
- `dbo:notableWork`
- `dbr:Elizabeth_II`
- `dbo:residence`
- `dbo:parent`

Exercise 10 DBpedia: Ask, Describe and Construct Queries

Go to <https://dbpedia.org/sparql>. Try the following queries:

1. Was Natalie Portman born in (some city of) the US?
2. Is Rome the capital of Italy?
3. Describe Rome

4. Describe 20 people born in Italy
5. Construct the graph of the ancestors of `dbo:Elizabeth_II`
6. Construct an RDF graph about characters from “A Song of Ice and Fire” (or some other series you like) and their relatives (and/or some other properties). or Read [here](#) about the `values` keyword.

A few useful IRIs:

- | | |
|------------------------------------|---|
| • <code>dbr:Natalie_Portman</code> | • <code>dbo:parent</code> |
| • <code>dbo:birthPlace</code> | • <code>dbo:relative</code> |
| • <code>dbo:country</code> | • <code>dbo:series</code> |
| • <code>dbr:United_States</code> | • <code>dbr:A_Song_of_Ice_and_Fire</code> |
| • <code>dbo:capital</code> | |

Exercise 11 DBpedia: Filters

Lists of functions for reference:

- On RDF terms: <https://www.w3.org/TR/sparql11-query/#func-rdfTerms>
- On strings: <https://www.w3.org/TR/sparql11-query/#func-strings>
- On numerics <https://www.w3.org/TR/sparql11-query/#func-numeric>
- On dates and times <https://www.w3.org/TR/sparql11-query/#func-date-time>

Go to <https://dbpedia.org/sparql>. Try the following queries:

1. Retrieve the names of 100 people born after 1984
2. Retrieve writer names (in English), the title of their works (in Italian) and their page number, but only if the page number is either less than 100 or over 9000.
3. Retrieve movies directed by Alfred Hitchcock that do not star James Stewart.
4. Retrieve movies directed by Alfred Hitchcock that do not star James Stewart and Grace Kelly.
5. Retrieve movies directed by Alfred Hitchcock that do not star James Stewart or Grace Kelly.

A few useful IRIs:

- `dbo:starring`
- `dbr:James_Stewart`

Exercise 12 Assignment and Aggregate Queries

1. Find out the number of books written by each writer (hint: group by writer, and use the aggregate function `COUNT`). Sort the results by number of books written .
2. Find out the average movie runtime by genre. Sort the results by runtime.
3. Find out how many movies are there (in DBpedia) by genre. Sort the results by number of movies.

4. Find out how many movies are there (in DBpedia) by country. Sort the results by number of movies
5. Find out the countries where movies directed by Alfred Hitchcock were made, and how many movies per country.

A few useful IRIs:

- `dbp:author`
- `dbp:genre`
- `dbo:runtime`
- `dbp:country`

Acknowledgements

- The reification puzzle of Fig. 2 was taken from [this set of slides](#) by Werner Nutt.
- Some of the queries were taken or adapted from Blerina Spahiu's Exercises in RDF & SPARQL for the Data Semantics course of the University of Milano-Bicocca.